

PERA-SAM

Predictive Equipment Reliability & Acoustics
Sound Analysis Manager

Software User Manual

IEEE Std 1063-2001 Compliant

Document ID: PERA-SAM-UM-2026-001

Version: 2.0.0

Date: 2026-09-24

Status: Release

Prepared By

Invictus-Team29

Faculty of Engineering, University of Peradeniya

Revision History

Version	Date	Author	Description
1.0.0	2026-01-15	Invictus-Team29	Initial draft
1.1.0	2026-06-01	Invictus-Team29	Added mobile app documentation
2.0.0	2026-09-24	Invictus-Team29	Full IEEE 1063 compliance; v2.0 features

Contents

Revision History	1
1 Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Intended Audience	4
1.4 Definitions, Acronyms, and Abbreviations	4
1.5 References	4
1.6 Document Conventions	5
2 System Overview	5
2.1 System Description	5
2.2 System Architecture	5
2.3 System Components	6
2.4 User Roles and Permissions	6
3 Getting Started	6
3.1 Prerequisites	6
3.2 Hardware Requirements	6
3.3 Software Requirements	6
3.4 Installation Procedures	7
3.4.1 ML Backend Installation	7
3.4.2 Mobile Application Installation	7
3.5 Initial Configuration	8
3.5.1 ML Backend Configuration	8
3.5.2 MIMII Dataset Setup	8
3.6 Starting the System	8
3.6.1 Starting the ML Backend	9
3.6.2 Starting the Mobile Application	9
3.7 Verifying Installation	9
3.7.1 ML Backend Verification	9
3.7.2 Mobile Application Verification	9
4 Using the Software — Mobile Application	10
4.1 Login Screen	10
4.2 Registration Screen	10
4.3 Dashboard (Home)	11
4.4 Analysis Screen	11
4.5 Web Dashboard — Sound Analysis Interface	12
4.6 Map — Service Providers	13
4.7 Repair Requests	13
4.8 Chat — Real-Time Messaging	13
4.9 Analysis History	14
4.10 Profile Screen	14
5 Using the Software — ML Backend API	14

5.1	API Overview	14
5.2	Endpoints Reference	15
5.2.1	GET / — API Status	15
5.2.2	GET /health — Health Check	15
5.2.3	GET /models — Available Models	15
5.2.4	POST /analyze — Analyze Audio File	15
5.3	Interpreting Analysis Results	16
5.3.1	Status Classification	16
5.3.2	Key Metrics Explained	16
5.4	Health Scoring System	16
6	Error Messages and Troubleshooting	17
6.1	Mobile Application Errors	17
6.2	ML Backend Errors	17
6.3	Network and Connectivity Issues	17
6.4	Known Limitations	17
7	Maintenance and Configuration	18
7.1	Backend Configuration	18
7.1.1	Changing the Server Port	18
7.1.2	Disabling Auto-Training	18
7.2	Model Retraining	18
7.3	Docker Deployment	18
7.4	Backup and Recovery	19
8	Safety and Security Considerations	19
8.1	Authentication and Authorization	19
8.2	Data Security	19
8.3	Network Security	19
8.4	Operational Safety	19
9	Appendices	19
	Appendix A: API Response Schema	19
	Appendix B: Machine Category Reference	20
	Appendix C: Feature Extraction Parameters	20
	Appendix D: Glossary	21
	Appendix E: Test Results	21

List of Figures

2.1	PERA-SAM System Architecture	5
4.1	PERA-SAM Login Screen — Web Application	10
4.2	PERA-SAM Dashboard — Overview of Sound Analysis Activity	11
4.3	PERA-SAM Sound Analysis Screen — Audio Upload and Equipment Selection	11
4.4	Web Dashboard — Sound Analysis Results	12
4.5	PERA-SAM Find Service Providers — Map View with Provider Listings	13
4.6	Repair Request Status Lifecycle	13
4.7	PERA-SAM Analysis History — Browsing Previous Diagnostic Reports	14
9.1	Web Dashboard — Sound Analysis Test Results	21

List of Tables

1.1	Intended Audience	4
1.2	Definitions, Acronyms, and Abbreviations	4
1.3	Document Conventions	5
2.1	Supported Machine Categories	5
2.2	System Components	6
2.3	User Roles and Permissions	6
3.1	ML Backend Server Hardware Requirements	6
3.2	Mobile Device Requirements	6
3.3	ML Backend Software Requirements	6
3.4	Key Python Dependencies	7
3.5	Mobile Application Software Requirements	7
4.1	Login Screen Elements	10
4.2	Login Error Messages	10
4.3	Registration Screen Elements	10
4.4	Dashboard Elements	11
4.5	Notification Types	11
4.6	Analysis Screen Elements	12
4.7	Status Color Interpretation	12
4.8	Map Screen Elements	13
4.9	Request Status Meanings	13
4.10	Analysis History Detail Fields	14
4.11	System Status Indicators	14
5.1	Analysis Modes	16
5.2	Status Classification Rules	16
5.3	Analysis Metrics	16
5.4	Health Scoring Formulas	16
5.5	Health Interpretation Guide	16
6.1	Mobile Application Errors	17
6.2	ML Backend Errors	17
6.3	Network and Connectivity Issues	17
7.1	Docker Configuration Parameters	18
7.2	Backup Assets	19
9.1	Machine Category Reference	20
9.2	Feature Extraction Parameters	20

1 Introduction

1.1 Purpose

This document serves as the official software user manual for **PERA-SAM** (Predictive Equipment Reliability & Acoustics — Sound Analysis Manager), version 2.0.0. It provides comprehensive instructions for installing, configuring, and operating the PERA-SAM system.

This manual is prepared in accordance with **IEEE Std 1063-2001**, *IEEE Standard for Software User Documentation*, and addresses all mandatory sections specified by the standard.

1.2 Scope

This manual covers:

- Installation and configuration of all PERA-SAM system components.
- Operation of the PERA-SAM mobile application (iOS & Android).
- Usage of the ML Backend API for sound anomaly detection.
- Interpretation of analysis results, health scores, and recommendations.
- Troubleshooting common issues.
- System maintenance and administration tasks.

This manual does **not** cover:

- Internal machine learning model architecture or training theory.
- Supabase cloud infrastructure administration.
- Source code modification or software development practices.

1.3 Intended Audience

Table 1.1: Intended Audience

Audience	Description
Field Technicians	Personnel who perform machine health inspections using acoustic analysis.
Maintenance Engineers	Engineers responsible for predictive maintenance schedules.
Facility Managers	Managers overseeing fleet-wide machine health and service requests.
System Administrators	IT staff responsible for deploying and maintaining the PERA-SAM backend.
Service Providers	Third-party repair companies interfacing with PERA-SAM for repair requests.

1.4 Definitions, Acronyms, and Abbreviations

Table 1.2: Definitions, Acronyms, and Abbreviations

Term	Definition
PERA-SAM	Predictive Equipment Reliability & Acoustics — Sound Analysis Manager
FFT	Fast Fourier Transform — transforms time-domain signals to frequency-domain
MFCC	Mel-Frequency Cepstral Coefficients — audio feature representation
Mel Spectrogram	A spectrogram where frequencies are converted to the Mel scale
MSE	Mean Squared Error — reconstruction error used by the autoencoder
AUC	Area Under the Curve — metric measuring discriminative ability
RUL	Remaining Useful Life — predicted operational lifespan of equipment
MIMII	Malfunctioning Industrial Machine Investigation and Inspection (Hitachi dataset)
API	Application Programming Interface
JWT	JSON Web Token — used for authentication
WAV	Waveform Audio File Format
Autoencoder	Neural network trained to reconstruct input; anomalies yield high reconstruction error

1.5 References

1. IEEE Std 1063-2001, *IEEE Standard for Software User Documentation*.
2. Purohit, H., Tanabe, R., Ichige, K., et al. “MIMII Dataset: Sound Dataset for Malfunctioning Industrial Machine Investigation and Inspection.” *arXiv:1909.09347*, 2019.
3. FastAPI Documentation — <https://fastapi.tiangolo.com/>
4. Expo Documentation — <https://docs.expo.dev/>
5. Supabase Documentation — <https://supabase.com/docs>
6. Librosa Documentation — <https://librosa.org/doc/latest/>
7. TensorFlow Documentation — https://www.tensorflow.org/api_docs

1.6 Document Conventions

Table 1.3: Document Conventions

Convention	Meaning
monospace text	Code, commands, file paths, API endpoints, or configuration values
Bold text	UI element labels, button names, or important terms
<i>Italic text</i>	Emphasis or first use of a defined term
> prefix	Terminal/shell commands
⚠	Warning — action may cause data loss or system instability
📝	Informational note
✓	Successful outcome or confirmation

2 System Overview

2.1 System Description

PERA-SAM is a centralized acoustic management system designed to listen to the “heartbeat” of machines. Traditional maintenance is reactive — fixing things only after they break. PERA-SAM shifts this to a **predictive** model by processing acoustic signatures using FFT and MFCC. The system detects subtle frequency shifts caused by friction, imbalances, or wear *before* catastrophic failure occurs.

Key Capabilities

1. **AI Sound Doctor** — Captures audio to generate a real-time health score for industrial assets.
2. **Predictive Maintenance** — Estimates machine health to prevent unexpected downtime.
3. **Self-Learning (Human-in-the-Loop)** — A feedback mechanism where technicians validate AI predictions.
4. **Service Provider Discovery** — Location-based search for nearby repair services.
5. **Repair Request Management** — End-to-end workflow for requesting and tracking repairs.
6. **Real-Time Chat** — Messaging system between asset owners and service providers.

Supported Machine Categories

Table 2.1: Supported Machine Categories

Category	Examples
Fan	Cooling fans, server fans, industrial fans
Pump	Hydraulic pumps, water pumps
Slider	Rail sliders, linear actuators
Valve	Industrial valves, control valves

2.2 System Architecture

The PERA-SAM system follows a three-tier architecture as shown in Figure 2.1.

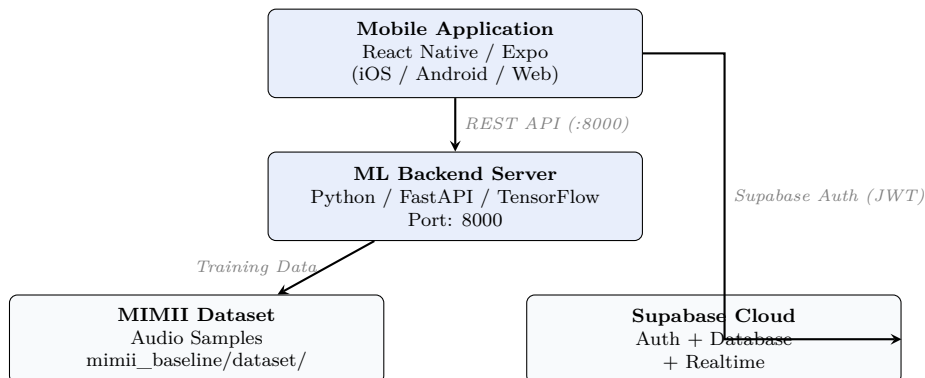


Figure 2.1: PERA-SAM System Architecture

2.3 System Components

Table 2.2: System Components

Component	Directory	Technology	Purpose
Mobile App	pera-sam-mobile/	React Native, Expo, TS	User-facing mobile application
ML Backend	model/server/	Python, FastAPI, TF	Sound analysis & anomaly detection API
Research Lab	mimii_baseline/	Python, Keras, Librosa	Original Hitachi research code & dataset
Cloud Services	Supabase (hosted)	PostgreSQL, GoTrue Auth	Authentication, data persistence, realtime

2.4 User Roles and Permissions

Table 2.3: User Roles and Permissions

Role	Capabilities
Standard User	Login, register, upload audio, view analysis results, view history, find service providers, submit repair requests, chat with service providers
Service Provider	Receive repair requests, accept/decline requests, communicate via chat, mark requests as completed
System Administrator	Configure ML backend, retrain models, manage deployment, view server logs

3 Getting Started

3.1 Prerequisites

Before installing PERA-SAM, ensure you have the following:

1. A modern computer (Windows 10+, macOS 12+, or Linux) for the ML backend.
2. A smartphone (iOS 13+ or Android 8+) or emulator for the mobile app.
3. An active internet connection for Supabase cloud services.
4. Administrative/root access on the host machine for the ML backend.

3.2 Hardware Requirements

Table 3.1: ML Backend Server Hardware Requirements

Resource	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	4 GB	8+ GB
Storage	2 GB (without dataset)	10+ GB (with full MIMII dataset)
GPU	Not required	Optional (NVIDIA CUDA)

Table 3.2: Mobile Device Requirements

Resource	Minimum
iOS	13.0 or later
Android	8.0 (API 26) or later
Storage	100 MB free space
Microphone	Required for live recording

3.3 Software Requirements

Table 3.3: ML Backend Software Requirements

Software	Version	Purpose
Python	3.9+	Runtime environment
pip	Latest	Package manager
Docker (opt.)	20.10+	Container deployment

Key Python packages (installed automatically via `requirements.txt`):

Table 3.4: Key Python Dependencies

Package	Version	Purpose
FastAPI	0.115.0	REST API framework
uvicorn	0.30.6	ASGI server
TensorFlow	2.17.0	Neural network inference
Librosa	0.10.2	Audio feature extraction
NumPy	1.26.4	Numerical computation
scikit-learn	1.5.2	Machine learning utilities
PyYAML	6.0.2	Configuration file parsing

Table 3.5: Mobile Application Software Requirements

Software	Version	Purpose
Node.js	18+	JavaScript runtime
npm	9+	Package manager
Expo CLI	Latest (via npx)	React Native build tooling
Expo Go	Latest	Mobile development client

3.4 Installation Procedures

3.4.1 ML Backend Installation

Step 1: Clone the repository.

```
1 > git clone https://github.com/cepdnaclk/e22-co2060-PERA-SAM.git
2 > cd e22-co2060-PERA-SAM
```

Listing 3.1: Clone repository

Step 2: Create and activate a Python virtual environment.

```
1 # Windows
2 > cd model\server
3 > python -m venv venv
4 > venv\Scripts\activate
5
6 # macOS / Linux
7 > cd model/server
8 > python3 -m venv venv
9 > source venv/bin/activate
```

Listing 3.2: Virtual environment setup

Step 3: Install Python dependencies.

```
1 > pip install -r requirements.txt
```

Listing 3.3: Install dependencies

✓ **Verification:** Run `pip list` and confirm that `fastapi`, `tensorflow`, and `librosa` are listed.

3.4.2 Mobile Application Installation

Step 1: Navigate to the mobile app directory.

```
1 > cd pera-sam-mobile
```

Step 2: Install Node.js dependencies.

```
1 > npm install
```

Step 3: Configure environment variables.

```
1 > cp .env.example .env
```

Step 4: Edit the `.env` file with your credentials:

```
1 # Supabase Configuration
2 EXPO_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
3 EXPO_PUBLIC_SUPABASE_ANON_KEY=your_anon_key_here
4
5 # ML Backend API URL
6 # Physical device: use your computer's local IP
```

```

7 # Android emulator: http://10.0.2.2:8000
8 # iOS simulator: http://localhost:8000
9 EXPO_PUBLIC_ML_API_URL=http://localhost:8000

```

Listing 3.4: Environment configuration (.env)

Note

If Supabase is not configured, the application will automatically operate in **Demo Mode** with a local mock session.

3.5 Initial Configuration

3.5.1 ML Backend Configuration

The ML backend is configured through `model/server/config.yaml`:

```

1 # Dataset Configuration
2 dataset:
3   base_directory: ../../mimii_baseline/dataset
4   noise_level: 0dB
5   pickle_directory: ../../mimii_baseline/pickle
6
7 # Feature Extraction (must match MIMII baseline exactly)
8 feature:
9   n_mels: 64
10  frames: 5
11  n_fft: 1024
12  hop_length: 512
13  power: 2.0
14
15 # Model Training
16 training:
17   epochs: 50
18   batch_size: 512
19   shuffle: true
20   validation_split: 0.1
21   calibration_files: 30
22
23 # Server
24 server:
25   host: "0.0.0.0"
26   port: 8000
27   auto_train: true

```

Listing 3.5: config.yaml — ML Backend Configuration

Warning

Do not modify the `feature` section unless you also retrain all models. Feature parameters must exactly match the training configuration.

3.5.2 MIMII Dataset Setup

If you have the MIMII dataset:

1. Download the dataset from the Hitachi MIMII repository.
2. Extract it to `mimii_baseline/dataset/`.
3. The expected directory structure is:

```

1 mimii_baseline/dataset/
2   0dB/
3     fan/
4       id_00/
5         normal/
6           00000000.wav, ...
7         abnormal/
8           00000000.wav, ...
9       id_02/
10      ...
11     pump/
12     slider/
13     valve/

```

Listing 3.6: MIMII Dataset directory structure

3.6 Starting the System

3.6.1 Starting the ML Backend

```
1 > cd model/server
2 > python main.py
```

Listing 3.7: Start ML backend

Upon startup, the following automatic sequence occurs:

1. **Load config.yaml** — configuration is parsed.
2. **Check for existing models** in `assets/` directory.
3. **Auto-train** if `auto_train: true` and models are missing (may take several minutes).
4. **Load models** into memory via the `SoundAnalyzer` inference engine.
5. **Start FastAPI server** on the configured host and port.

Expected console output on successful startup:

```
1 =====
2          PERA-SAM ML Backend v2.0
3      python main.py  ->  handles everything
4 =====
5 Starting on http://0.0.0.0:8000
6 [OK] All models present - skipping training
7 [OK] Inference engine ready - 6 model(s) loaded
```

Listing 3.8: Successful startup output

3.6.2 Starting the Mobile Application

```
1 > cd pera-sam-mobile
2 > npx expo start
```

Listing 3.9: Start mobile application

This launches the Expo development server. You can then:

- **Scan the QR code** with the Expo Go app on your mobile device.
- **Press a** to open on an Android emulator.
- **Press i** to open on an iOS simulator.
- **Press w** to open in a web browser.

3.7 Verifying Installation

3.7.1 ML Backend Verification

Open a web browser and navigate to `http://localhost:8000`. Expected response:

```
1 {
2   "status": "Online",
3   "message": "PERA-SAM ML Sound Analysis API",
4   "version": "2.0.0",
5   "models_loaded": 6,
6   "capabilities": [
7     "Auto-Train on Startup",
8     "Anomaly Detection",
9     "Per-Model Calibrated Thresholds",
10    "Health Scoring",
11    "Multi-Machine Identification"
12  ]
13 }
```

Listing 3.10: API Status Response

3.7.2 Mobile Application Verification

1. Launch the mobile app.
2. The **Login Screen** should appear with the PERA-SAM branding.
3. If Supabase is not configured, enter any email and password — the app will operate in **Demo Mode**.
4. The **Dashboard** screen should load, showing the greeting and quick-action cards.

4 Using the Software — Mobile Application

4.1 Login Screen

The Login Screen is the entry point to the PERA-SAM application. Figure 4.1 shows the web-based login interface.

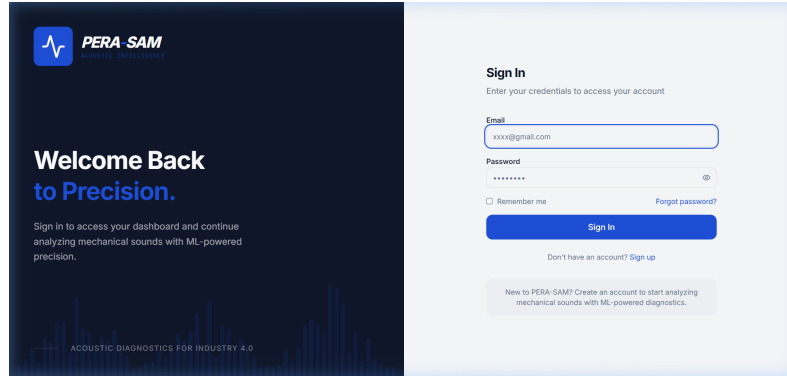


Figure 4.1: PERA-SAM Login Screen — Web Application

Table 4.1: Login Screen Elements

Element	Description
Email Field	Enter your registered email address.
Password Field	Enter your account password.
Show/Hide Password	Toggle button (eye icon) to reveal or conceal the password.
Remember Me	Checkbox to persist the session across app restarts.
Sign In Button	Submits the login credentials for authentication.
Create Account Link	Navigates to the Registration screen.
Demo Mode Banner	Displayed when Supabase is not configured; any credentials are accepted.

Procedure — Logging In

1. Enter your **email address** in the Email field.
2. Enter your **password** in the Password field.
3. Optionally, check **Remember Me**.
4. Tap **Sign In**.
5. Upon successful authentication, you are redirected to the **Dashboard**.

Error Handling

Table 4.2: Login Error Messages

Error	Cause	Resolution
“Missing Fields”	Email or password is empty	Fill in both fields
“Login Failed”	Invalid credentials	Check your email and password
“Network request failed”	Cannot reach Supabase	Check internet connection

4.2 Registration Screen

New users must register before using the system.

Table 4.3: Registration Screen Elements

Element	Description
Full Name Field	Your display name used across the application.
Email Field	Must be a valid email address.
Password Field	Minimum 6 characters. A strength indicator bar provides visual feedback.
Confirm Password Field	Must exactly match the password.
Password Strength Bar	Color-coded bar: Red (< 6 chars), Amber (6–7 chars), Green (8+ chars).
Create Account Button	Submits the registration form.
Sign In Link	Returns to the Login screen.

Procedure — Creating an Account

1. Enter your **Full Name**.
2. Enter your **Email Address**.
3. Enter a **Password** (minimum 6 characters). Observe the strength indicator.
4. Re-enter your password in the **Confirm Password** field.
5. Tap **Create Account**.
6. Check your email inbox for a **confirmation link** (if Supabase email verification is enabled).
7. After confirming, return to the Login screen and sign in.

4.3 Dashboard (Home)

The Dashboard is the primary screen after login, providing a real-time overview of machine health status. Figure 4.2 shows the web dashboard with analysis summary cards, recent analyses, quick actions, and AI system status.

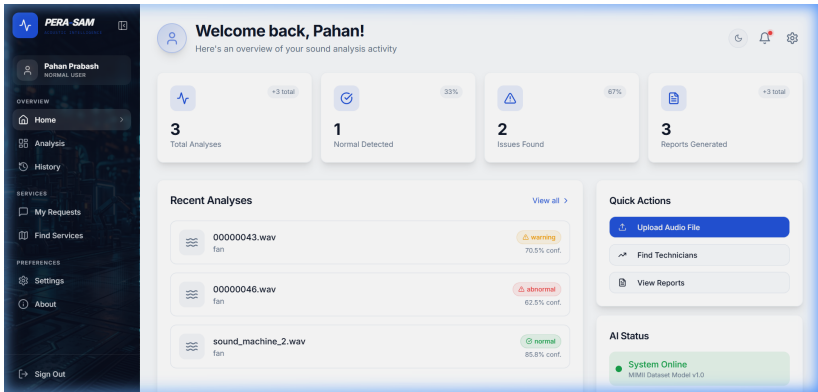


Figure 4.2: PERA-SAM Dashboard — Overview of Sound Analysis Activity

Table 4.4: Dashboard Elements

Element	Description
Greeting Header	Displays “Hello, [Name]” with a personalized welcome.
Notification Bell	Opens the notification modal showing recent alerts.
Quick Actions Card	Shortcut to “Start Analysis” and “View History”.
Recent Analyses	List of the 5 most recent analysis records with status indicators.
Pull-to-Refresh	Swipe down to reload dashboard data.

Table 4.5: Notification Types

Type	Icon	Description
Anomaly Alert		A recent analysis detected an anomaly or warning condition.
Message		New chat message from a service provider.
Repair Update		Status change on a repair request.
System		General system notifications.

4.4 Analysis Screen

The Analysis Screen is the core feature of PERA-SAM, where users submit audio samples for anomaly detection. Figure 4.3 shows the web-based analysis interface with audio upload, equipment details, and the Analyze Sound button.

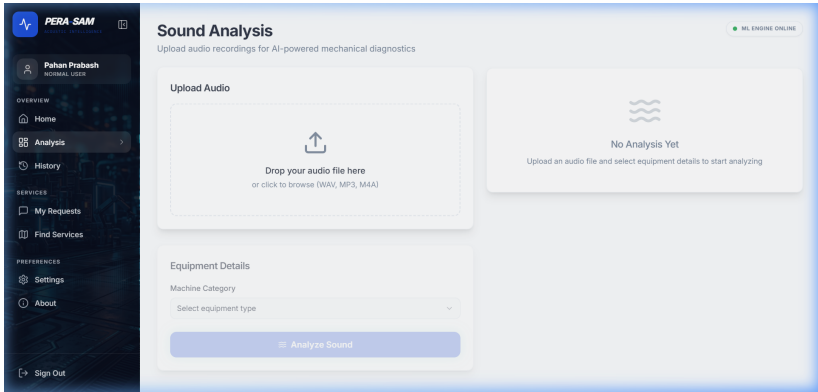


Figure 4.3: PERA-SAM Sound Analysis Screen — Audio Upload and Equipment Selection

Table 4.6: Analysis Screen Elements

Element	Description
Machine Category Selector	Choose the machine type: Fan, Pump, Slider, Valve, or Vehicle Bearing.
Upload Audio Button	Opens the device file picker to select a .wav audio file.
Record Audio Button	Starts/stops live audio recording using the device microphone.
Recording Indicator	Pulsing red dot and timer displayed during active recording.
Selected File Info	Shows filename, size, and source of the selected audio.
Analyze Button	Sends the selected audio to the ML Backend for processing.
Results Panel	Displays the analysis outcome after processing.
Save to History Button	Persists the result to the analysis history in Supabase.

Procedure — Analyzing Machine Sound

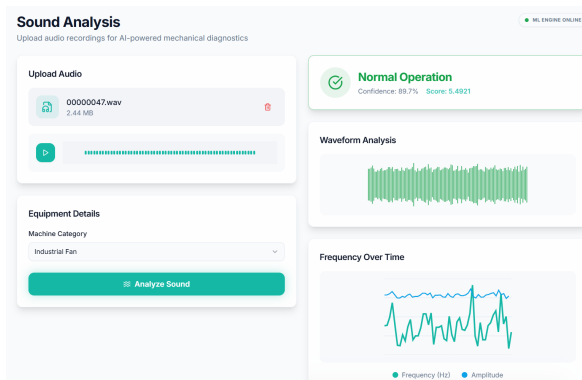
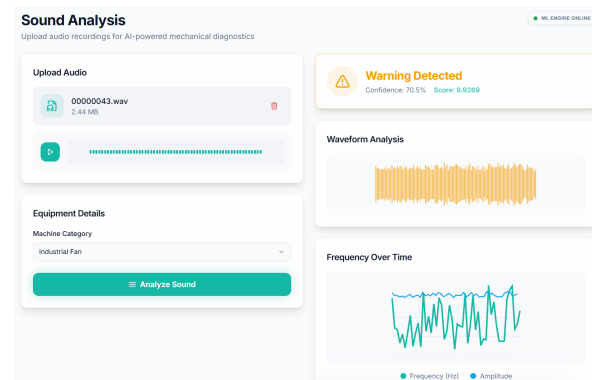
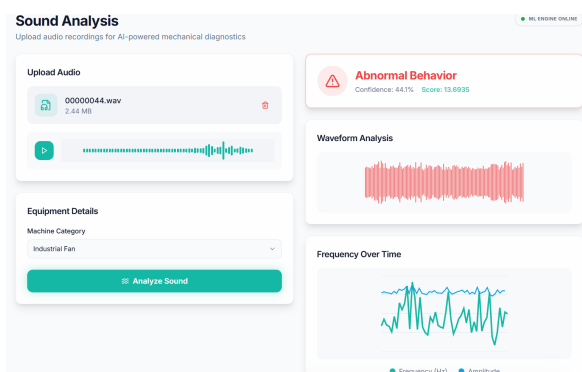
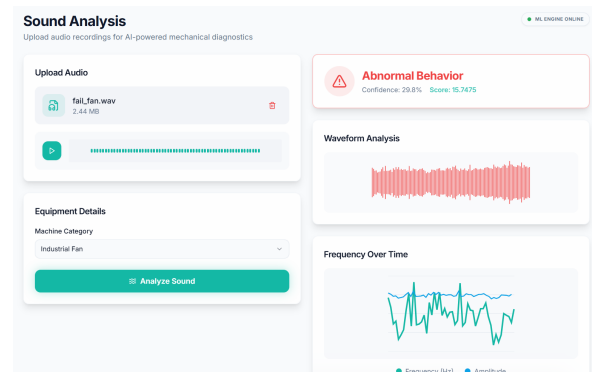
1. Navigate to the **Analysis** tab (microphone icon in the bottom bar).
2. **Select a Machine Category** from the horizontal category list.
3. Provide audio input via one of two methods:
 - **Method A — Upload:** Tap **Upload Audio File**. Select a **.wav** file from your device storage.
 - **Method B — Record:** Tap **Record Audio**. Grant microphone permission. Position the device near the machine. Tap **Stop Recording** when sufficient audio is captured.
4. Tap the **Analyze** button.
5. Wait for the analysis to complete (a loading spinner is displayed).
6. Review the results (see Table 4.7).
7. Optionally tap **Save to History** to persist the result.

Table 4.7: Status Color Interpretation

Status	Color	Meaning
✓ Normal	Green	Machine operating within normal parameters. No action required.
⚠ Warning	Amber	Unusual patterns detected. Schedule maintenance soon.
✗ Anomaly	Red	Machine failure may be imminent. Stop operation and inspect immediately.

4.5 Web Dashboard — Sound Analysis Interface

The web dashboard provides the same analysis capabilities with a richer visual interface. Figure 4.4 shows the three possible analysis outcomes.

**(a) Normal Operation — Health score 89.7%****(b) Warning Detected — Confidence 70.5%****(c) Abnormal Behavior — Confidence 44.1%****(d) Abnormal Behavior — fail_fan.wav****Figure 4.4:** Web Dashboard — Sound Analysis Results

4.6 Map — Service Providers

The Map screen helps users discover nearby repair and maintenance service providers. Figure 4.5 shows the Find Services interface with an interactive map, category filters, and a list of nearby service providers with ratings.

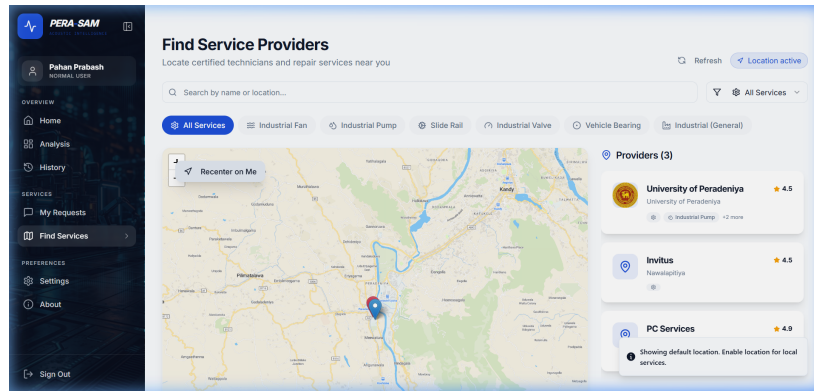


Figure 4.5: PERA-SAM Find Service Providers — Map View with Provider Listings

Table 4.8: Map Screen Elements

Element	Description
Search Bar	Search service providers by name or keyword.
Category Filters	Horizontal filter chips: All, Fan, Pump, Slider, Valve, Bearing, General.
Provider List	Scrollable list of service providers sorted by distance.
Provider Card	Displays name, address, rating, distance, availability, and contact.
Request Repair	Initiates a repair request to the selected provider.
Call Button	Opens the phone dialer with the provider's number.
Get Directions	Opens the native maps app with directions.

4.7 Repair Requests

The Requests screen manages the lifecycle of repair requests between users and service providers.

Request Status Lifecycle

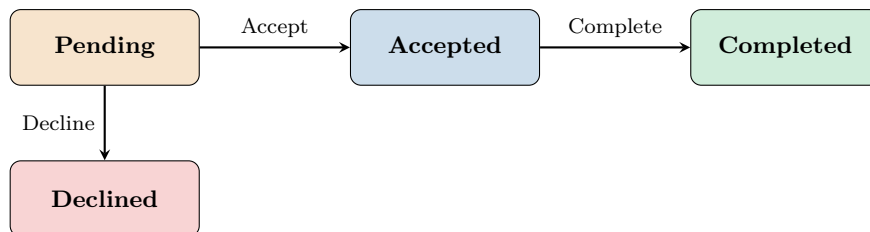


Figure 4.6: Repair Request Status Lifecycle

Table 4.9: Request Status Meanings

Status	Color	Description
Pending	Amber	Request submitted; awaiting provider response.
Accepted	Blue	Provider has accepted and work is in progress.
Completed	Green	Repair has been completed successfully.
Declined	Red	Provider has declined the request.

4.8 Chat — Real-Time Messaging

The Chat screen provides real-time communication between users and service providers within the context of a repair request.

Features:

- Messages are **color-coded** by sender (your messages on the right in indigo, received messages on the left in white).
- **Real-time updates** via Supabase PostgreSQL change subscriptions.
- **Read status tracking** for unread message indicators.
- **Unread badge** on the Requests tab icon showing the count of unread messages.

Procedure — Sending a Message

1. From the **Requests** screen, tap the **Chat** button on any request.
2. Type your message in the input field at the bottom.
3. Tap the **Send** button (paper plane icon).
4. The message appears instantly in the conversation.

4.9 Analysis History

The History screen provides a comprehensive log of all past analyses performed by the user. Figure 4.7 shows the analysis history with summary statistics, search/filter capabilities, and individual analysis records with status badges.

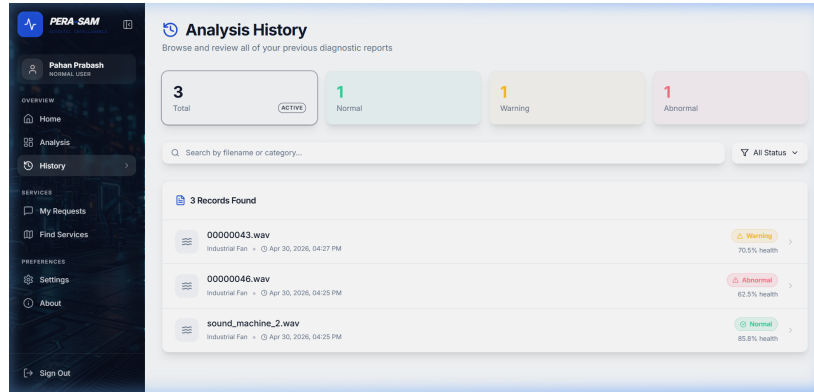


Figure 4.7: PERA-SAM Analysis History — Browsing Previous Diagnostic Reports

Table 4.10: Analysis History Detail Fields

Field	Description
Category	Machine type analyzed
Machine ID	Specific machine model matched
Status	Normal / Warning / Anomaly
Confidence	Model prediction confidence percentage
Anomaly Score	Raw reconstruction error value
Recommendation	Suggested action based on the analysis
Filename	Original audio file name
Analyzed On	Timestamp of the analysis

4.10 Profile Screen

The Profile screen displays user account information, system configuration status, and technology stack details.

Table 4.11: System Status Indicators

Indicator	Color	Meaning
✓ Connected	Green	Service is reachable and configured.
🔌 Demo Mode	Amber	Supabase not configured; using mock data.
✗ Error	Red	Service is unreachable or misconfigured.

5 Using the Software — ML Backend API

5.1 API Overview

The PERA-SAM ML Backend exposes a RESTful API built with FastAPI. It provides endpoints for health checking, model listing, and audio analysis.

- **Base URL:** `http://<host>:8000`
- **Authentication:** The API currently does not require authentication. Access control should be managed at the network level.
- **Request Content-Type:** `multipart/form-data` (for file uploads)
- **Response Content-Type:** `application/json`

5.2 Endpoints Reference

5.2.1 GET / — API Status

Returns the current status of the ML backend.

```

1 GET http://localhost:8000/
2
3 Response:
4 {
5   "status": "Online",
6   "message": "PERA-SAM ML Sound Analysis API",
7   "version": "2.0.0",
8   "models_loaded": 6,
9   "capabilities": [
10    "Auto-Train on Startup",
11    "Anomaly Detection",
12    "Per-Model Calibrated Thresholds",
13    "Health Scoring",
14    "Multi-Machine Identification"
15  ]
16 }
```

Listing 5.1: API Status — Request and Response

5.2.2 GET /health — Health Check

```

1 GET http://localhost:8000/health
2
3 Response: { "status": "ok" }
```

Listing 5.2: Health Check Endpoint

5.2.3 GET /models — Available Models

```

1 GET http://localhost:8000/models
2
3 Response:
4 {
5   "categories": {
6     "fan": ["00", "02", "04", "06"],
7     "pump": ["00", "02", "04", "06"],
8     "slider": ["00", "02", "04", "06"],
9     "valve": ["00", "02", "04", "06"]
10  },
11   "metrics_available": [...]
12 }
```

Listing 5.3: Models Endpoint

5.2.4 POST /analyze — Analyze Audio File

The primary endpoint. Uploads a WAV audio file and receives an anomaly detection result.

```

1 POST http://localhost:8000/analyze
2 Content-Type: multipart/form-data
3
4 Fields:
5   file: (binary)      -- Required. The WAV audio file.
6   category: string    -- Optional. Machine category.
7   machine_id: string  -- Optional. Specific machine ID.
```

Listing 5.4: Analyze Endpoint — Request

Example using cURL:

```

1 > curl -X POST http://localhost:8000/analyze \
2   -F "file=@machine_recording.wav" \
3   -F "category=fan" \
4   -F "machine_id=00"
```

Listing 5.5: cURL example

Table 5.1: Analysis Modes

Mode	Condition	Behavior
Direct Mode	Both <code>category</code> and <code>machine_id</code> provided	Uses the specified model with its calibrated threshold
Auto-Detect Mode	Only <code>category</code> provided	Scans all models in the category; picks the best match
Default Mode	Neither provided	Falls back to a general-purpose default model

5.3 Interpreting Analysis Results

5.3.1 Status Classification

The system classifies the result into one of three statuses based on the anomaly score relative to the calibrated threshold:

Table 5.2: Status Classification Rules

Status	Condition	Meaning
Normal	$\text{score} < \text{threshold} \times 0.8$	Machine operating within normal parameters.
Warning	$\text{threshold} \times 0.8 \leq \text{score} \leq \text{threshold}$	Unusual patterns detected; degradation beginning.
Anomaly	$\text{score} > \text{threshold}$	Significant deviation from normal; potential failure.

5.3.2 Key Metrics Explained

Table 5.3: Analysis Metrics

Metric	Range	Description
<code>score</code>	$0 - \infty$	Mean Squared Error between input and autoencoder reconstruction. Lower is healthier.
<code>max_frame_error</code>	$0 - \infty$	Maximum per-frame MSE. High values indicate transient spikes.
<code>min_frame_error</code>	$0 - \infty$	Minimum per-frame MSE. Represents the “cleanest” portion.
<code>threshold_used</code>	$0 - \infty$	The calibrated decision boundary.
<code>health_percentage</code>	$0 - 100$	Human-readable health score.
<code>model_auc</code>	$0 - 1.0$	Area Under the Curve. Higher = more reliable.
<code>detection_reliability</code>	Text	High ($\text{AUC} > 0.85$), Medium ($0.7-0.85$), Low (< 0.7).

5.4 Health Scoring System

The health percentage is calculated as follows:

Table 5.4: Health Scoring Formulas

Condition	Formula	Health Range
$s < T \times 0.8$	$100 - \frac{s}{T \times 0.8} \times 15$	85%–100%
$T \times 0.8 \leq s \leq T$	$85 - \frac{s - T \times 0.8}{T \times 0.2} \times 15$	70%–85%
$s > T$	$\max(0, 70 - \frac{s - T}{T} \times 70)$	0%–70%

Where s = anomaly score and T = calibrated threshold.

Table 5.5: Health Interpretation Guide

Health %	Engine Health	Recommended Action
85%–100%	Good	No action needed. Continue normal operation.
70%–85%	Degrading	Schedule preventive maintenance within the next window.
0%–70%	Critical	URGENT: Stop operation immediately. Inspect the machine.

6 Error Messages and Troubleshooting

6.1 Mobile Application Errors

Table 6.1: Mobile Application Errors

Error Message	Probable Cause	Resolution
“Missing Fields”	Form fields are empty	Fill in all required fields
“Weak Password”	Password < 6 characters	Use ≥ 6 characters
“Password Mismatch”	Confirm password mismatch	Re-enter the confirmation password
“Login Failed”	Invalid credentials	Verify email and password
“Network request failed”	Supabase URL unreachable	Check internet; verify EXPO_PUBLIC_SUPABASE_URL
“ML API not configured”	EXPO_PUBLIC_ML_API_URL missing	Add URL to .env; restart Expo
“Analysis failed”	ML backend error	Check backend is running; verify audio file
“Audio too short”	Insufficient audio duration	Re-record or upload longer sample ($\geq 0.5s$)

6.2 ML Backend Errors

Table 6.2: ML Backend Errors

Error / Log Message	Probable Cause	Resolution
config.yaml not found	Configuration file missing	Create config.yaml in model/server/
No .h5 models in assets/	No trained models	Ensure dataset exists; set auto_train: true
Trainer error (non-fatal)	Training failed	Check dataset paths in config.yaml
Analysis failed for <fn>	Audio processing error	Ensure valid WAV file
Audio too short	File too short	Use audio ≥ 0.5 seconds
No models for category	Missing category model	Train models or omit category parameter

6.3 Network and Connectivity Issues

Table 6.3: Network and Connectivity Issues

Issue	Resolution
Mobile app cannot reach ML Backend	Physical device: Use computer’s local IP (e.g., <code>http://192.168.x.x:8000</code>). Android emulator: Use <code>http://10.0.2.2:8000</code> . iOS simulator: Use <code>http://localhost:8000</code> . Allow port 8000 through Windows Firewall.
Supabase connection refused	Verify EXPO_PUBLIC_SUPABASE_URL and anon key; ensure project is active.
Real-time subscriptions not working	Enable Supabase Realtime for the <code>request_messages</code> and <code>analysis_results</code> tables.
CORS errors in web mode	The ML backend allows all origins (<code>allow_origins=["*"]</code>). Check browser console.

6.4 Known Limitations

1. **Audio Format:** Only WAV format files are supported. MP3, AAC, and other formats must be converted before analysis.
2. **Recording Quality:** Live recordings may have lower signal quality. Results vary based on microphone quality and ambient noise.
3. **Machine Categories:** Currently trained on fan, pump, slider, and valve categories from the MIMII dataset.
4. **Offline Mode:** Requires internet for Supabase features. Analysis works offline if the ML backend is on the same local network.
5. **Concurrent Users:** The ML backend serves requests sequentially. For production, consider horizontal scaling.
6. **Python Version:** TensorFlow 2.17 does not support Python 3.13+. Use Python 3.12 or earlier (`py -3.12 main.py`).

7 Maintenance and Configuration

7.1 Backend Configuration

7.1.1 Changing the Server Port

Edit `model/server/config.yaml`:

```
1 server:
2   port: 9000 # Change from 8000 to 9000
```

Alternatively, set the `PORT` environment variable:

```
1 > PORT=9000 python main.py
```

Warning

Update `EXPO_PUBLIC_ML_API_URL` in the mobile app's `.env` to match the new port.

7.1.2 Disabling Auto-Training

For production deployments where models are pre-trained:

```
1 server:
2   auto_train: false
```

7.2 Model Retraining

To retrain models with updated or new data:

1. **Update the dataset** — Add new WAV files to the appropriate `normal/` and `abnormal/` subdirectories in the MIMII dataset.
2. **Clear existing models** — Delete the `.h5` files from `model/server/assets/`.
3. **Clear the pickle cache** — Delete cached features from `mimii_baseline/pickle/`.
4. **Restart the server** with `auto_train: true`:

```
1 > python main.py
```

The system will detect missing models and initiate the full training pipeline: Feature extraction → Pickle caching → Autoencoder training (50 epochs) → Threshold calibration → Model export.

7.3 Docker Deployment

Build the Docker image:

```
1 > cd model/server
2 > docker build -t pera-sam-backend .
```

Run the container:

```
1 > docker run -d \
2   --name pera-sam \
3   -p 8000:8000 \
4   -v $(pwd)/assets:/app/assets \
5   pera-sam-backend
```

Table 7.1: Docker Configuration Parameters

Parameter	Default	Description
Exposed Port	8000	The API listening port
Health Check Interval	30s	Time between health probes
Health Check Timeout	10s	Maximum wait for health response
Start Period	60s	Grace period for model loading/training
Retries	5	Failed checks before marking unhealthy

7.4 Backup and Recovery

Table 7.2: Backup Assets

Asset	Location	Importance	Notes
Trained Models	model/server/assets/*.h5	Critical	Avoids retraining (hours of compute)
Model Metrics	model/server/assets/metrics.yaml	Critical	Contains calibrated thresholds
Configuration	model/server/config.yaml	High	System configuration
Pickle Cache	mimii_baseline/pickle/	Medium	Can be regenerated from dataset
Server Logs	model/server/server.log	Low	Operational audit trail

Recovery Procedure:

1. Restore the .h5 model files and metrics.yaml to model/server/assets/.
2. Restore config.yaml to model/server/.
3. Restart the server: `python main.py`.
4. The system will detect existing models and skip training.

8 Safety and Security Considerations

8.1 Authentication and Authorization

- User authentication is managed by **Supabase GoTrue** using email/password credentials and JWT tokens.
- Session tokens expire and must be refreshed; the mobile app handles this automatically.
- In Demo Mode (Supabase not configured), **no real authentication is performed**. Do not use Demo Mode in production.

8.2 Data Security

- Audio files uploaded for analysis are stored **temporarily** on the server and **deleted immediately** after processing.
- Analysis results stored in Supabase are associated with the authenticated user's ID and are only accessible to that user.
- The ML API allows all CORS origins by default (`allow_origins=["*"]`). For production, restrict this to known domains.

8.3 Network Security

- The ML Backend does **not** use HTTPS by default. For production deployments, place the API behind a reverse proxy (e.g., Nginx) with TLS/SSL.
- Supabase connections use HTTPS by default.

8.4 Operational Safety

✖ Important Disclaimer

PERA-SAM provides **advisory recommendations only**. The system's analysis results should **never** be the sole basis for critical safety decisions. Always follow:

1. Manufacturer-recommended maintenance schedules.
2. Established safety protocols and standard operating procedures.
3. Professional engineering judgment.

The system's predictions are probabilistic and may produce false positives (alerting when no issue exists) or false negatives (missing actual issues). The detection reliability metric indicates the model's confidence level.

9 Appendices

Appendix A: API Response Schema

Full /analyze Response Schema:

```
1 {  
2   "filename": string,
```

```

3  "category_requested": string,
4  "machine_id_requested": string,
5  "analysis": {
6    "status": string,           // "Normal"|"Warning"|"Anomaly"
7    "score": float,            // Mean Squared Error
8    "max_frame_error": float,
9    "min_frame_error": float,
10   "threshold_used": float,
11   "health_percentage": float, // 0.0 - 100.0
12   "engine_health": string,    // "Good"|"Degrading"|"Critical"
13   "recommendation": string,
14   "machine_id": string,
15   "machine_category": string,
16   "model_auc": float,        // 0.0 - 1.0
17   "detection_reliability": string,
18   "baseline_compliant": bool,
19   "model_used": string
20 }
21 }

```

Listing 9.1: Complete /analyze Response Schema

Appendix B: Machine Category Reference

Table 9.1: Machine Category Reference

Category	ID	Description	Typical Faults Detected
Fan	fan	Cooling fans, server fans	Blade imbalance, bearing wear, obstruction, motor degradation
Pump	pump	Hydraulic pumps, water pumps	Cavitation, seal leakage, impeller damage, dry running
Slider	slider	Rail sliders, linear actuators	Rail scoring, lubrication failure, alignment issues
Valve	valve	Industrial valves, control valves	Seal degradation, blockage, actuator malfunction

Appendix C: Feature Extraction Parameters

These parameters are identical to the Hitachi MIMII baseline and **must not be changed** without retraining all models.

Table 9.2: Feature Extraction Parameters

Parameter	Value	Description
n_mels	64	Number of Mel frequency bands
n_fft	1024	FFT window size (samples)
hop_length	512	Hop length between frames (samples)
n_frames	5	Sliding window width for feature concatenation
power	2.0	Power spectrogram exponent
Log formula	$\frac{20.0}{\text{power}} \times \log_{10}(\text{mel} + \varepsilon)$	Log-Mel spectrogram conversion
Input dimension	320	$\text{n_mels} \times \text{n_frames} = 64 \times 5 = 320$

Autoencoder Architecture

Input(320) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(8, ReLU)
 $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Output(320, Linear)

The bottleneck layer (8 neurons) forces the network to learn a compressed representation of “normal” machine sounds. Anomalies produce higher reconstruction error because they differ from the learned normal patterns.

Appendix D: Glossary

Term	Definition
Anomaly Detection	The process of identifying data patterns that deviate significantly from expected behavior.
Autoencoder	Neural network used to learn efficient encodings; learns “normal” machine sound patterns.
Calibrated Threshold	Data-driven decision boundary computed from both normal and abnormal training samples.
FastAPI	Modern, high-performance Python web framework for building APIs.
Mel Scale	Perceptual scale of pitches approximating human auditory perception.
MSE	Mean squared difference between predicted and actual values; used as anomaly score.
Predictive Maintenance	Maintenance strategy using data analysis to predict equipment failures before they occur.
Reconstruction Error	Difference between autoencoder input and output; higher error indicates anomaly.
Spectrogram	Visual representation of the spectrum of frequencies in a signal as they vary with time.
Supabase	Open-source Firebase alternative for authentication, database, and realtime subscriptions.
WAV	Uncompressed audio file format preserving full signal fidelity.

Appendix E: Test Results — Sound Analysis Screenshots

The following screenshots demonstrate the web-based Sound Analysis interface with various test audio files and their classification results.

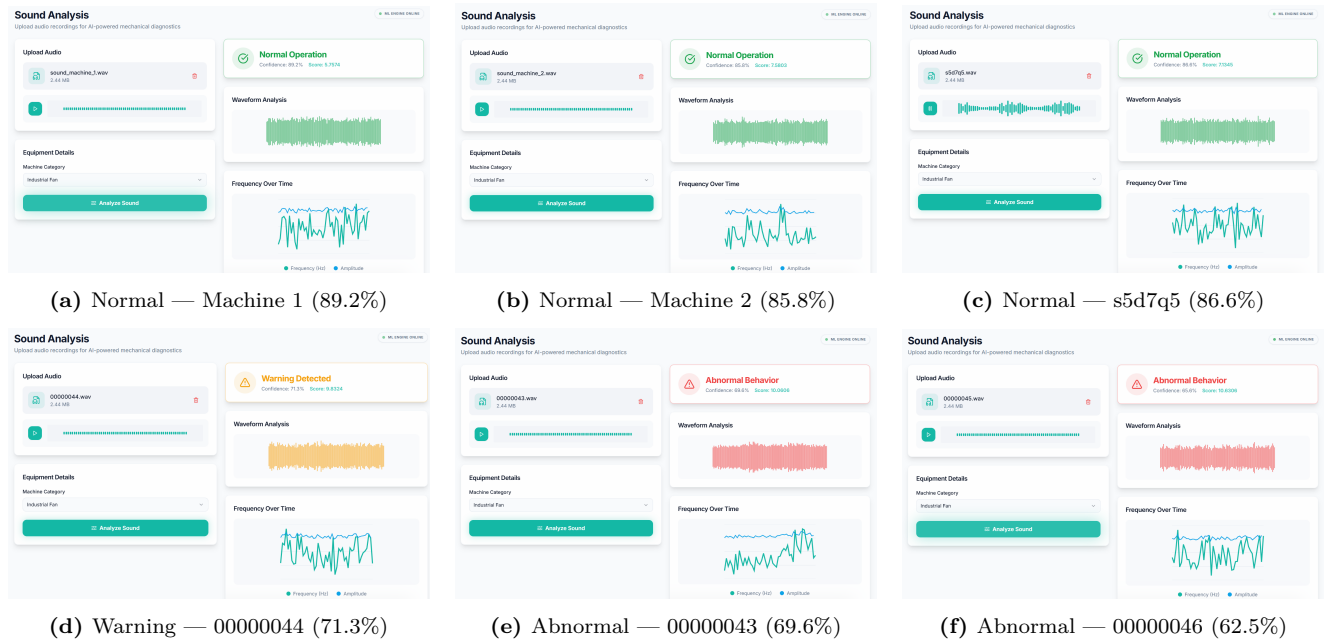


Figure 9.1: Web Dashboard — Sound Analysis Test Results